# Information Retrieval – Practical

## INDEX


| Sr. No. | Aim | Date | Page No. | Sign |
|---|---|---|---|---|
| 1 | **Document Indexing and Retrieval**<br>• Implement an inverted index construction algorithm.<br>• Build a simple document retrieval system using the constructed index. | 7/11/2025 | 02 | |
| 2 | **Retrieval Models**<br>• Implement the Boolean retrieval model and process queries.<br>• Implement the vector space model with TF-IDF weighting and cosine similarity. | 14/11/2025 | 04 | |
| 3 | **Spelling Correction in IR Systems**<br>• Develop a spelling correction module using edit distance algorithms.<br>• Integrate the spelling correction module into an information retrieval system. | 28/11/2025 | 08 | |
| 4 | **Evaluation Metrics for IR Systems**<br>• Calculate precision, recall, and F-measure for a given set of retrieval results.<br>• Use an evaluation toolkit to measure average precision and other evaluation metrics. | 05/12/2025 | 10 | |
| 5 | **Text Categorization**<br>• Implement a text classification algorithm (e.g., Naive Bayes or Support Vector Machines).<br>• Train the classifier on a labelled dataset and evaluate its performance. | 12/12/2025 | 12 | |
| 6 | **Clustering for Information Retrieval**<br>• Implement a clustering algorithm (e.g., K-means or hierarchical clustering).<br>• Apply the clustering algorithm to a set of documents and evaluate the clustering results. | 19/12/2025 | 17 | |
| 7 | **Web Crawling and Indexing**<br>• Develop a web crawler to fetch and index web pages.<br>• Handle challenges such as robots.txt, dynamic content, and crawling delays. | 03/01/2026 | 19 | |
| 8 | **Link Analysis and PageRank**<br>• Implement the PageRank algorithm to rank web pages based on link analysis.<br>• Apply the PageRank algorithm to a small web graph and analyze the results. | 17/01/2026 | 23 | |
| 9 | **Learning to Rank**<br>• Implement a learning to rank algorithm (e.g., RankSVM or RankBoost).<br>• Train the ranking model using labelled data and evaluate its effectiveness. | 31/01/2026 | 26 | |
| 10 | **Advanced Topics in Information Retrieval ·**<br>• Implement a text summarization algorithm (e.g., extractive or abstractive).<br>• Build a question-answering system using techniques such as information extraction | 13/02/2026 | 28 | |

# Practical No.1

## Aim: Document Indexing and Retrieval.

- Implement an inverted index construction algorithm.
- Build a simple document retrieval system using the constructed index.

**Theory:**

- An Inverted Index is a data structure used in information retrieval systems to efficiently retrieve documents or web pages containing a specific term or set of terms. In an inverted index, the index is organized by terms (words), and each term points to a list of documents or web pages that contain that term. Inverted indexes are widely used in search engines, database systems, and other applications where efficient text search is required. An inverted index is an index data structure storing a mapping from content, such as words or numbers, to its locations in a document or a set of documents.

**Input:**

```
import nltk

from nltk.corpus import stopwords

document1 = "The quick brown fox jumped over the lazy dog"

document2 = "The lazy dog slept in the sun"

nltk.download('stopwords')

stopWords = stopwords.words('english')

tokens1 = document1.lower().split()

tokens2 = document2.lower().split()

terms = list(set(tokens1 + tokens2))

inverted_index = {}

occ_num_doc1 = {}

occ_num_doc2 = {}

for term in terms:

    if term in stopWords:
```

```
        continue
    documents = []
    if term in tokens1:
        documents.append("Document 1")
        occ_num_doc1[term] = tokens1.count(term)
    if term in tokens2:
        documents.append("Document 2")
        occ_num_doc2[term] = tokens2.count(term)
    inverted_index[term] = documents
for term, documents in inverted_index.items():
    print(term, "->", end=" ")
    for doc in documents:
        if doc == "Document 1":
            print(f"{doc}({occ_num_doc1.get(term, 0)}),", end=" ")
        else:
            print(f"{doc}({occ_num_doc2.get(term, 0)}),", end=" ")
    print()
print("Performed by 251049 Omkar Parelkar")
```

## Output:-

```
jumped -> Document 1(1),
sun -> Document 2(1),
lazy -> Document 1(1), Document 2(1),
dog -> Document 1(1), Document 2(1),
fox -> Document 1(1),
slept -> Document 2(1),
quick -> Document 1(1),
brown -> Document 1(1),
Performed by 251049 Omkar Parelkar
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

# Practical No.2

**Aim: Retrieval Models**

- Implement the Boolean retrieval model and process queries.
- Implement the vector space model with TF-IDF weighting and cosine similarity.

**Theory:**

Boolean retrieval model is a simple retrieval model based on set theory and boolean algebra. Queries are designed as boolean expressions which have precise semantics. A vector space model is a model where each term of the query is considered a vector dimension.

The TF-IDF value increases proportionally to the frequency a word appears in a document (TF) and is offset by the number of documents in the corpus that also contain that word (IDF).

## Input (A):

```
documnets = {
    1: "apple banana orange",
    2: "apple banana",
    3: "banana orange",
    4: "apple"
}

def build_index(docs):
    index = {}
    for doc_id, text in docs.items():
        terms = set(text.split())
        for term in terms:
            if term not in index:
                index[term] = {doc_id}
            else:
```

```
            index[term].add(doc_id)
    return index

inverted_index = build_index(documnets)

def boolean_and(operands, index):
    if not operands:
        return list(range(1, len(documnets) + 1))
    result = index.get(operands[0], set())
    for term in operands[1:]:
        result = result.intersection(index.get(term, set()))
    return list(result)

def boolean_or(operands, index):
    result = set()
    for term in operands:
        result = result.union(index.get(term, set()))
    return list(result)

def boolean_not(operand, index, total_docs):
    operand_set = set(index.get(operand, set()))
    all_docs_set = set(range(1, total_docs + 1))
    return list(all_docs_set.difference(operand_set))

query1 = ["apple", "banana"]
query2 = ["apple", "orange"]

result1 = boolean_and(query1, inverted_index)
result2 = boolean_or(query2, inverted_index)
result3 = boolean_not("orange", inverted_index, len(documnets))
```

```
print("Documents containing 'apple' and 'banana':", result1)
print("Documents containing 'apple' or 'orange':", result2)
print("Documents not containing 'orange':", result3)
print("Performed by 251049 Omkar Parelkar")
```

## Output:-

```
Documents containing 'apple' and 'banana': [1, 2]
Documents containing 'apple' or 'orange': [1, 2, 3, 4]
Documents not containing 'orange': [2, 4]
Performed by 251049 Omkar Parelkar
```

## Input(B):-

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
import nltk
from nltk.corpus import stopwords
import numpy as np
from numpy.linalg import norm

train_set = ["The sky is blue.", "The sun is bright."]
test_set = ["The sun is the sky is bright."]

nltk.download('stopwords')
stopWords = stopwords.words('english')

vectorizer = CountVectorizer(stop_words=stopWords)
transformer = TfidfTransformer()

trainVectorizerArray = vectorizer.fit_transform(train_set).toarray()
testVectorizerArray = vectorizer.transform(test_set).toarray()
```

```
print('Fit Vectorizer to train set', trainVectorizerArray)
print('Transform Vectorizer to test set', testVectorizerArray)

cx = lambda a, b: round(np.inner(a, b) / (norm(a) * norm(b)), 3)

for vector in trainVectorizerArray:
    print(vector)

for testV in testVectorizerArray:
    print(testV)

    cosine = cx(vector, testV)
    print(cosine)

transformer.fit(trainVectorizerArray)
print()

tfidf = transformer.transform(testVectorizerArray)
print(tfidf.todense())
```

**Output:-**

```
Fit Vectorizer to train set [[1 0 1 0]
 [0 1 0 1]]
Transform Vectorizer to test set [[0 1 1 1]]
[1 0 1 0]
[0 1 0 1]
[0 1 1 1]
0.816

[[0.         0.57735027 0.57735027 0.57735027]]
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

# Practical No.3

**Aim: Spelling Correction in IR Systems**

Develop a spelling correction module using edit distance algorithms. Integrate the spelling correction module into an information retrieval system.

## Theory:

Spelling correction is a critical component of Information Retrieval(IR) systems, enhancing the accuracy and efficiency of searching and retrieving relevant documents. More advanced IR systems may use deep learning models to understand and predict corrections based on large datasets of text, effectively learning from patterns in user behavior.

## Input:

```
from optparse import Values


def editDistance(str1, str2, m, n):
    # If first string is empty
    if m == 0:
        return n

    # If second string is empty
    if n == 0:
        return m

    # If last characters are same, ignore last char and recur
    if str1[m - 1] == str2[n - 1]:
        return editDistance(str1, str2, m - 1, n - 1)

    # If last characters are different, consider all possibilities
    return 1 + min(
```

```
        editDistance(str1, str2, m, n - 1),      # Insert
        editDistance(str1, str2, m - 1, n),      # Remove
        editDistance(str1, str2, m - 1, n - 1)   # Replace
    )

str1 = "sunday"
str2 = "saturday"

print('Edit Distance is:', editDistance(str1, str2, len(str1), len(str2)))
```

## Output:-

```
Edit Distance is: 3
Performed by 251049 Omkar Parelkar
```

# Practical No.4

## Aim: Evaluation Metrics for IR Systems

Calculate precision, recall, and F-measure for a given set of retrieval results. Use an evaluation toolkit to measure average precision and other evaluation metrics.

## Theory:

Evaluation metrics are used to measure the performance of an IR system without requiring real-time user interaction. Precision measures the proportion of relevant documents retrieved from all retrieved documents, while recall measures the proportion of relevant documents retrieved from all relevant documents in the dataset.

## Input (A):

```
def calculate_metrics(retrieved_set, relevant_set):
    true_positive = len(retrieved_set.intersection(relevant_set))
    false_positive = len(retrieved_set.difference(relevant_set))
    false_negative = len(relevant_set.difference(retrieved_set))

    print(
        "True Positive: ", true_positive,
        "\nFalse Positive: ", false_positive,
        "\nFalse Negative: ", false_negative, "\n"
    )

    precision = true_positive / (true_positive + false_positive)
    recall = true_positive / (true_positive + false_negative)
    f_measure = 2 * precision * recall / (precision + recall)
```

```
    return precision, recall, f_measure

retrieved_set = set(["doc1", "doc2", "doc3"])
relevant_set = set(["doc1", "doc4"])

precision, recall, f_measure = calculate_metrics(retrieved_set, relevant_set)

print(f"Precision: {precision}")
print("Recall:", recall)
print("F-Measure:", f_measure)
```

## Output:-

```
True Positive:  1
False Positive:  2
False Negative:  1

Precision: 0.3333333333333333
Recall: 0.5
F-Measure: 0.4
```

## Input (B):

```
from sklearn.metrics import average_precision_score
y_true = [0, 1, 1, 0, 1, 1]
y_scores = [0.1, 0.4, 0.35, 0.8, 0.65, 0.9]
average_precision = average_precision_score(y_true, y_scores)
print(f'Average precision-recall score: {average_precision}')
```

## Output:-

```
Average precision-recall score: 0.8041666666666667
```

# Practical No.5

## Aim: Text Categorization

Implement a text classification algorithm (e.g., Naïve Bayes or Support Vector Machines). Train the classifier on a labelled dataset and evaluate its performance.

## Theory:

Text classification algorithms typically rely on converting text data into numerical features, using techniques like bag-of-words or TF-IDF. Two popular machine learning algorithms for text classification are Naïve Bayes classifier (NB) and Support Vector Machine (SVM).

## Input:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, accuracy_score

# Sample labeled dataset
data = {
    'text': [
        "This is a positive movie. I loved it!",
        "The acting was terrible. I hated it.",
        "A great film with an amazing plot.",
        "Boring and predictable. Waste of time.",
        "Absolutely fantastic! Highly recommend.",
        "The worst movie I've ever seen.",
        "A truly captivating story. Brilliant!",
        "So bad, I almost walked out of the theater.",
```

```
        "Excellent performance by the lead actor.",
        "Utterly disappointing and a complete failure.",
    ],
    'label': [
        'positive', 'negative', 'positive', 'negative',
        'positive', 'negative', 'positive', 'negative',
        'positive', 'negative'
    ]
}

df = pd.DataFrame(data)

# 1. Data Preprocessing (TF-IDF Vectorization)
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(df['text'])
y = df['label']

# 2. Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train the Naive Bayes Classifier
classifier = MultinomialNB()
classifier.fit(X_train, y_train)

# 4. Make Predictions
y_pred = classifier.predict(X_test)

# 5. Evaluate Performance
```

```
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print(f"Classification Report:\n{report}")

# Example of prediction on new data
new_texts = [
    "This was an amazing experience.",
    "I did not enjoy this at all."
]

new_texts_transformed = tfidf.transform(new_texts)
new_predictions = classifier.predict(new_texts_transformed)
print("Predictions for new texts:", new_predictions)

# Improved dataset
data_improved = {
    'text': [
        "This is a positive movie. I loved it!",
        "The acting was terrible. I hated it.",
        "A great film with an amazing plot.",
        "Boring and predictable. Waste of time.",
        "Absolutely fantastic! Highly recommend.",
        "The worst movie I've ever seen.",
        "A truly captivating story. Brilliant!",
        "So bad, I almost walked out of the theater.",
        "Excellent performance by the lead actor.",
        "Utterly disappointing and a complete failure.",
        "This movie was okay.",
```

```
        "I found it quite entertaining",
        "It left me wanting more",
        "It was a complete mess",
        "The story made no sense",
    ],
    'label': [
        'positive', 'negative', 'positive', 'negative',
        'positive', 'negative', 'positive', 'negative',
        'positive', 'negative', 'neutral', 'positive',
        'positive', 'negative', 'negative'
    ]
}

df_improved = pd.DataFrame(data_improved)

X_improved = tfidf.fit_transform(df_improved['text'])
y_improved = df_improved['label']

X_train_improved, X_test_improved, y_train_improved, y_test_improved = train_test_split(
    X_improved, y_improved, test_size=0.2, random_state=42
)

classifier_improved = MultinomialNB()
classifier_improved.fit(X_train_improved, y_train_improved)

y_pred_improved = classifier_improved.predict(X_test_improved)

accuracy_improved = accuracy_score(y_test_improved, y_pred_improved)
report_improved = classification_report(y_test_improved, y_pred_improved)
```

```
print("\nImproved Model Results:")
print(f"Accuracy: {accuracy_improved}")
print(f"Classification Report:\n{report_improved}")

new_texts_improved = [
    "This was an amazing experience.",
    "I did not enjoy this at all.",
    "This was just average."
]

new_texts_transformed_improved = tfidf.transform(new_texts_improved)
new_predictions_improved = classifier_improved.predict(new_texts_transformed_improved)

print("Predictions for new texts (improved):", new_predictions_improved)
```

# Output:

```
Accuracy: 0.0
Classification Report:
              precision    recall  f1-score   support

    negative       0.00      0.00      0.00       1.0
    positive       0.00      0.00      0.00       1.0

    accuracy                           0.00       2.0
   macro avg       0.00      0.00      0.00       2.0
weighted avg       0.00      0.00      0.00       2.0

Predictions for new texts: ['positive' 'positive']

Improved Model Results:
Accuracy: 0.3333333333333333
Classification Report:
              precision    recall  f1-score   support

    negative       0.33      1.00      0.50         1
    positive       0.00      0.00      0.00         2

    accuracy                           0.33         3
   macro avg       0.17      0.50      0.25         3
weighted avg       0.11      0.33      0.17         3

Predictions for new texts (improved): ['positive' 'negative' 'negative']
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
```

# Practical No.6

## **Aim:** Clustering for Information Retrieval

- Implement a clustering algorithm (e.g., K-means or hierarchical clustering).
- Apply the clustering algorithm to a set of documents and evaluate the clustering results.

## Theory:

- Clustering groups similar documents together, which helps organize large datasets and improves navigation by enabling users to explore related content easily.
- Clustering can reduce the complexity of data by summarizing large datasets into representative clusters, allowing for more efficient processing and analysis in IR tasks.

## Input:

```
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.cluster import KMeans



documents = [

    "Cats are known for their agility and grace",

    "Dogs are often called man's best friend.",

    "Some dogs are train to assist people with disabilities.",

    "The sun rises in the east and sets in the west.",

    "Many cats enjoy climbing trees and chasing toys."

]



vectorizer = TfidfVectorizer(stop_words='english')

X = vectorizer.fit_transform(documents)
```

```
kmeans = KMeans(n_clusters=3, random_state=0).fit(X)
print(kmeans.labels_)
```

## Output:-

```
[0 2 0 1 0]
```

# Practical No.7

## Aim: Web Crawling and Indexing

- Develop a web crawler to fetch and index web pages.
- Handle challenges such as robot.txts, dynamic content, and crawling delays.

## Theory:

- Web crawling is the automated process of systematically browsing the internet to discover and collect web pages.
- Indexing is the process of organizing and storing the collected data in a way that allows for efficient retrieval.

## Input:

```
import requests

from bs4 import BeautifulSoup

import time

from urllib.parse import urljoin, urlparse

from urllib.robotparser import RobotFileParser


def get_html(url):

    headers = {

        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko)'

    }

    try:

        response = requests.get(url, headers=headers)

        response.raise_for_status()

        return response.text

    except requests.exceptions.HTTPError as errh:

        print(f"HTTP Error: {errh}")

    except requests.exceptions.RequestException as err:

        print(f"Request Error: {err}")
```

```
    return None

def save_robots_txt(url):
    try:
        robots_url = urljoin(url, '/robots.txt')
        robots_content = get_html(robots_url)
        if robots_content:
            with open('robots.txt', 'wb') as file:
                file.write(robots_content.encode('utf-8-sig'))
    except Exception as e:
        print(f"Error saving robots.txt: {e}")

def load_robots_txt():
    try:
        with open('robots.txt', 'rb') as file:
            return file.read().decode('utf-8-sig')
    except FileNotFoundError:
        return None

def extract_links(html, base_url):
    soup = BeautifulSoup(html, 'html.parser')
    links = []
    for link in soup.find_all('a', href=True):
        absolute_url = urljoin(base_url, link['href'])
        links.append(absolute_url)
    return links

def is_allowed_by_robots(url, robots_content):
    parser = RobotFileParser()
    parser.parse(robots_content.split('\n'))
```

```
    return parser.can_fetch('*', url)

def crawl(start_url, max_depth=3, delay=1):
    visited_urls = set()

    def recursive_crawl(url, depth, robots_content):
        if (
            depth > max_depth
            or url in visited_urls
            or not is_allowed_by_robots(url, robots_content)
        ):
            return

        visited_urls.add(url)
        time.sleep(delay)

        html = get_html(url)
        if html:
            print(f"Crawling {url}")
            links = extract_links(html, url)
            for link in links:
                recursive_crawl(link, depth + 1, robots_content)

    save_robots_txt(start_url)
    robots_content = load_robots_txt()

    if not robots_content:
        print("Unable to retrieve robots.txt. Crawling without restrictions.")

    recursive_crawl(start_url, 1, robots_content)
```

# Example usage:

print("Performed by ty bsc cs 251049 Omkar Parelkar")

crawl('https://wikipedia.com', max_depth=2, delay=2)

## Output:-

```
Performed by ty bsc cs 251049 Omkar Parelkar
Crawling https://wikipedia.com
Crawling https://en.wikipedia.org/
Crawling https://ja.wikipedia.org/
Crawling https://de.wikipedia.org/
Crawling https://ru.wikipedia.org/
Crawling https://fr.wikipedia.org/
Crawling https://es.wikipedia.org/
Crawling https://it.wikipedia.org/
Crawling https://zh.wikipedia.org/
Crawling https://pt.wikipedia.org/
Crawling https://pl.wikipedia.org/
Crawling https://ar.wikipedia.org/
Crawling https://fa.wikipedia.org/
Crawling https://arz.wikipedia.org/
Crawling https://nl.wikipedia.org/
Crawling https://ceb.wikipedia.org/
Crawling https://sv.wikipedia.org/
Crawling https://uk.wikipedia.org/
Crawling https://vi.wikipedia.org/
Crawling https://war.wikipedia.org/
Crawling https://af.wikipedia.org/
Crawling https://sq.wikipedia.org/
```

# Practical No.8

## **Aim:** Link Analysis and PageRank

- Implement the PageRank algorithm to rank web pages based on link analysis.
- Apply the PageRank algorithm to a small web graph and analyze the results.

## Theory:

- Link analysis refers to the examination of hyperlinks between web pages to evaluate their relationships and significance.
- PageRank assigns a numerical score to each page, representing its importance.

## Input:

```
import numpy as np


def page_rank(graph, damping_factor=0.85, max_iterations=100, tolerance=1e-6):
    # Get the number of nodes
    num_nodes = len(graph)

    # Initialize PageRank values
    page_rank = np.ones(num_nodes) / num_nodes

    # Iterative PageRank calculation
    for _ in range(max_iterations):
        prev_page_ranks = np.copy(page_rank)

        for node in range(num_nodes):
            # Calculate the contribution from incoming links
            incoming_links = [
                i for i, neighbors in enumerate(graph) if node in neighbors
```

```
            ]

            if not incoming_links:
                continue

            page_rank[node] = (
                (1 - damping_factor) / num_nodes
                + damping_factor
                * sum(
                    prev_page_ranks[link] / len(graph[link])
                    if len(graph[link]) else 0
                    for link in incoming_links
                )
            )

        # Check for convergence
        if np.linalg.norm(page_rank - prev_page_ranks, 2) < tolerance:
            break

    return page_rank

# Example usage
if __name__ == "__main__":
    web_graph = [[1, 2], [0, 2], [0, 1], [1, 2]]

    # Calculate PageRank
    result = page_rank(web_graph)

    for i, pr in enumerate(result):
        print(f"Page {i}: {pr}")
```

**Output:-**

```
Page 0: 0.6725117940472367
Page 1: 0.7470731975560085
Page 2: 0.7470731975560085
Page 3: 0.25
```

# Practical No.9

## **Aim:** Learning to Rank

- Implement a learning to rank algorithm (e.g., RankSVM or RankBoost).
- Train the ranking model using labelled data and evaluate its effectiveness.

## Theory:

- Learning to rank (LTR) algorithms are machine learning techniques aimed at improving the ranking of items—like web pages, documents, or products—based on their relevance to user queries.
- RankBoost is a learning-to-rank algorithm that utilizes boosting techniques to improve the ranking of items in information retrieval tasks.

## Input:

```
import numpy as np


def page_rank(graph, damping_factor=0.85, max_iterations=100, tolerance=1e6):
    num_nodes = len(graph)
    page_ranks = np.ones(num_nodes) / num_nodes

    for _ in range(max_iterations):
        prev_page_ranks = np.copy(page_ranks)

        for node in range(num_nodes):
            incoming_links = [i for i, v in enumerate(graph) if node in v]

            if not incoming_links:
                continue
```

```
            page_ranks[node] = (
                (1 - damping_factor) / num_nodes
                + damping_factor
                * sum(prev_page_ranks[link] / len(graph[link]) for link in incoming_links)
            )

        if np.linalg.norm(page_ranks - prev_page_ranks, 2) < tolerance:
            break

    return page_ranks

if __name__ == "__main__":
    web_graph = [[1, 2], [0, 2], [0, 1], [1, 2]]

    result = page_rank(web_graph)

    for i, pr in enumerate(result):
        print(f"Page {i}: {pr}")
```

**Output:-**

```
Page 0: 0.25
Page 1: 0.35624999999999996
Page 2: 0.35624999999999996
Page 3: 0.25
```

# Practical No.10

## **Aim:** Advanced Topics in Information Retrieval

- Implement a text summarization algorithm (e.g., extractive or abstractive).
- Build a question-answering system using techniques such as information extraction.

## Theory:

- Automatic Text Summarization is a key technique in Natural Language Processing (NLP) that uses algorithms to reduce large texts while preserving essential information.
- Extractive summarization algorithms automatically generate summaries by selecting and combining key passages from the original text. · Abstractive summarization generates entirely new sentences to convey key ideas from the original text.

## Input (A):

```
!pip install sumy


from IPython import get_ipython

from IPython.display import display


# Import necessary libraries

from sumy.parsers.plaintext import PlaintextParser

from sumy.nlp.tokenizers import Tokenizer

from sumy.summarizers.lsa import LsaSummarizer


# Download the 'punkt_tab' data for NLTK

import nltk

nltk.download('punkt_tab')  # This line downloads the necessary data


# Input text to be summarized
```

```
input_text = """
"""

parser = PlaintextParser.from_string(input_text, Tokenizer("english"))

# Create an LSA summarizer
summarizer = LsaSummarizer()

# Generate the summary
summary = summarizer(
    parser.document,
    sentences_count=3  # You can adjust the number of sentences in the summary
)

# Output the summary
print("Original Text:")
print(input_text)

print("\nSummary:")
for sentence in summary:
    print(sentence)
```

# Output:-

```
Collecting sumy
  Downloading sumy-0.11.0-py2.py3-none-any.whl.metadata (7.5 kB)
Collecting docopt<0.7,>=0.6.1 (from sumy)
  Downloading docopt-0.6.2.tar.gz (25 kB)
  Preparing metadata (setup.py) ... done
Collecting breadability>=0.1.20 (from sumy)
  Downloading breadability-0.1.20.tar.gz (32 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: requests>=2.7.0 in /usr/local/lib/python3.12/dist-packages (from sumy) (2.32.4)
Collecting pycountry>=18.2.23 (from sumy)
  Downloading pycountry-24.6.1-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: nltk>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from sumy) (3.9.1)
Requirement already satisfied: chardet in /usr/local/lib/python3.12/dist-packages (from breadability>=0.1.20->sumy) (5.2.0)
Requirement already satisfied: lxml>=2.0 in /usr/local/lib/python3.12/dist-packages (from breadability>=0.1.20->sumy) (6.0.2)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk>=3.0.2->sumy) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk>=3.0.2->sumy) (1.5.2)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk>=3.0.2->sumy) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk>=3.0.2->sumy) (4.67.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (2025.11.12)
Downloading sumy-0.11.0-py2.py3-none-any.whl (97 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 97.3/97.3 kB 4.7 MB/s eta 0:00:00
Downloading pycountry-24.6.1-py3-none-any.whl (6.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.3/6.3 MB 34.7 MB/s eta 0:00:00
Building wheels for collected packages: breadability, docopt
  Building wheel for breadability (setup.py) ... done
```

```
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk>=3.0.2->sumy) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk>=3.0.2->sumy) (4.67.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.7.0->sumy) (2025.11.12)
Downloading sumy-0.11.0-py2.py3-none-any.whl (97 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 97.3/97.3 kB 4.7 MB/s eta 0:00:00
Downloading pycountry-24.6.1-py3-none-any.whl (6.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.3/6.3 MB 34.7 MB/s eta 0:00:00
Building wheels for collected packages: breadability, docopt
  Building wheel for breadability (setup.py) ... done
  Created wheel for breadability: filename=breadability-0.1.20-py2.py3-none-any.whl size=21695 sha256=273c46839164839746666631f052de0a03b0d328a3689ce15d3cc175e13a601ad
  Stored in directory: /root/.cache/pip/wheels/32/99/64/59305409cacd03aa03e7bddf31a9db34b1fa7033bd41972662
  Building wheel for docopt (setup.py) ... done
  Created wheel for docopt: filename=docopt-0.6.2-py2.py3-none-any.whl size=13706 sha256=c6b503c2d0db87261fd662a0b81487070d1558d8c4deeeea45f23be49645d23c
  Stored in directory: /root/.cache/pip/wheels/1a/bf/a1/4cee4f7678c68c5875ca89eaccf460593539805c3906722228
Successfully built breadability docopt
Installing collected packages: docopt, pycountry, breadability, sumy
Successfully installed breadability-0.1.20 docopt-0.6.2 pycountry-24.6.1 sumy-0.11.0
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Original Text:

Your input text goes here. It can be a long paragraph or multiple paragraphs.


Summary:
Your input text goes here.
It can be a long paragraph or multiple paragraphs.
```

# Input (B):

```
import spacy


# Load the spaCy model
nlp = spacy.load("en_core_web_sm")


# Sample text corpus
text_corpus = """
The Eiffel Tower is a wrought-iron lattice tower on the Champ de Mars in Paris, France.
It is named after the engineer Gustave Eiffel, whose company designed and built the tower.
The tower is 330 meters tall and was completed in 1889.
```

```
"""

# Process the text with spaCy
doc = nlp(text_corpus)

# Extract entities (e.g., names, dates, locations)
entities = [(ent.text, ent.label_) for ent in doc.ents]
print("Extracted Entities:", entities)

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Preprocess the text and extract sentences
sentences = [sent.text for sent in doc.sents]

# Create a TF-IDF vectorizer
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(sentences)

def answer_question(question):
    # Vectorize the question
    question_vec = vectorizer.transform([question])

    # Compute cosine similarity between the question and each sentence
    similarities = cosine_similarity(question_vec, X)

    # Find the most similar sentence
    most_similar_idx = similarities.argmax()
    most_similar_sentence = sentences[most_similar_idx]
```

```
    return most_similar_sentence

# Example usage
question = "How tall is the Eiffel Tower?"
answer = answer_question(question)
print("Answer:", answer)

def enhanced_answer_question(question):
    # Vectorize the question
    question_vec = vectorizer.transform([question])

    # Compute cosine similarity between the question and each sentence
    similarities = cosine_similarity(question_vec, X)

    # Find the most similar sentence
    most_similar_idx = similarities.argmax()
    most_similar_sentence = sentences[most_similar_idx]

    # Extract entities from the most similar sentence
    doc = nlp(most_similar_sentence)
    entities = [(ent.text, ent.label_) for ent in doc.ents]

    # Return the sentence and relevant entities
    return most_similar_sentence, entities

# Example usage
question = "Who designed the Eiffel Tower?"
answer, entities = enhanced_answer_question(question)

print("Answer:", answer)
```

```
print("Relevant Entities:", entities)
```

## Output:-

```
Extracted Entities: [('The Eiffel Tower', 'LOC'), ('the Champ de Mars', 'FAC'), ('Paris', 'GPE'), ('France', 'GPE'), ('Gustave Eiffel', 'PERSON'), ('330 meters', 'QUANTITY'),
Answer: The tower is 330 meters tall and was completed in 1889.

Answer: It is named after the engineer Gustave Eiffel, whose company designed and built the tower.

Relevant Entities: [('Gustave Eiffel', 'PERSON')]
```